▶ 작업 보고서: Spring Boot 4.0 업데이트 및 AOP 유효성 검사 적용

- **작업 일시**: 2026-03-18
- **진행 단계**: 완료

1. 🌊 전체 작업 흐름 (Workflow)

1. **환경 최신화**: `build.gradle`에서 Spring Boot 버전을 4.0.3으로 업데이트하고 Java 21 기반의 최신 개발 환경을 구축했습니다.
2. **검증 의존성 추가**: 데이터 유효성 검사를 위한 `spring-boot-starter-validation` 라이브러리를 프로젝트에 포함했습니다.
3. **회원가입 DTO 설계**: 사용자로부터 입력받을 아이디, 비밀번호, 이메일에 대한 제약 조건(길이, 형식 등)을 `UserRequest.Join` 클래스에 정의했습니다.
4. **AOP 핸들러 구현**: 모든 컨트롤러에서 공통적으로 발생하는 유효성 검사 에러 처리 로직을 `MyValidationHandler`라는 하나의 관점(Aspect)으로 분리하여 구현했습니다.
5. **API 적용 및 검증**: 회원가입 API 엔드포인트를 생성하고, 실제 빌드를 통해 Spring Boot 4.0 환경에서의 정상 작동을 확인했습니다.

2. 🧩 핵심 코드 (Core Logic)

```
// UserRequest.Join - 유효성 검사 규칙 정의
@Data
public static class Join {
    @NotBlank(message = "유저네임은 필수입니다")
    @Size(min = 4, max = 20, message = "유저네임은 4자에서 20자 사이여야 합니다")
    private String username;

    @NotBlank(message = "비밀번호는 필수입니다")
    @Size(min = 4, max = 20, message = "비밀번호는 4자에서 20자 사이여야 합니다")
    private String password;

    @Email(message = "올바른 이메일 형식이 아닙니다")
    private String email;
}

// MyValidationHandler - AOP를 이용한 공통 에러 처리
@Around("execution(* com.example.demo..*Controller.*(..))")
public Object validationAdvice(ProceedingJoinPoint jp) throws Throwable {
    Object[] args = jp.getArgs();
    for (Object arg : args) {
        // 컨트롤러 메서드의 인자 중 BindingResult가 있다면 에러를 확인합니다.
        if (arg instanceof BindingResult bindingResult) {
            if (bindingResult.hasErrors()) {
                var error = bindingResult.getFieldErrors().get(0);
                return Resp.fail(HttpStatus.BAD_REQUEST,
error.getDefaultMessage());
            }
```

```
            }
        }
        return jp.proceed(); // 에러가 없다면 원래 메서드를 실행합니다.
    }
```

3. 🎈 상세비유 (Easy Analogy)

이번 작업은 **"놀이공원 입구의 자동 검문소"**를 설치한 것과 같습니다.

- 기존에는 각 놀이기구(컨트롤러 메서드)마다 직원이 서서 키나 나이를 일일이 확인해야 했습니다. (중복 코드 발생)
- 이제는 놀이공원 정문(AOP)에 자동 검문소를 세워, 규정에 맞지 않는 사람(유효성 검사 실패)은 아예 입장시키지 않고 입구에서 바로 돌려보냅니다. 덕분에 각 놀이기구 직원들은 오직 운행(비즈니스 로직)에만 집중할 수 있게 되었습니다!

4. 📚 기술 딥다이브 (Technical Deep-dive)

- **Spring Boot 4.0**: 최신 Java 25 및 Spring Framework 7의 기능을 지원하며, 모듈화가 강화되어 더욱 빠르고 가벼운 애플리케이션 실행 환경을 제공합니다.
- **AOP (Aspect Oriented Programming)**: 여러 곳에 흩어져 있는 '공통 관심사(예: 유효성 검사, 로깅)'를 하나의 모듈로 묶어서 관리하는 기술입니다. 이를 통해 핵심 로직의 가독성을 높이고 유지보수를 편리하게 만듭니다.
- **Jakarta Validation**: `@Valid`, `@NotBlank` 등의 어노테이션을 사용하여 데이터의 형식을 선언적으로 검증하는 표준 기술입니다. 복잡한 `if` 문 없이도 데이터의 무결성을 보장할 수 있습니다.